

# 第二讲

## bits, byte and Integers

笔记整理自 2026 年 9 月 10 日课程

### 1. Before class

The sequence of the textbook and the class is a bit different. Use the slides to follow the class and carefully review the slides right after class. Then you can read the textbook and pick the parts you need.<sup>1</sup>

### 2. Binary Representations

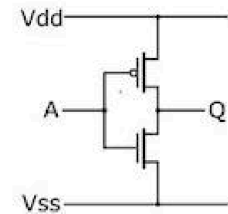
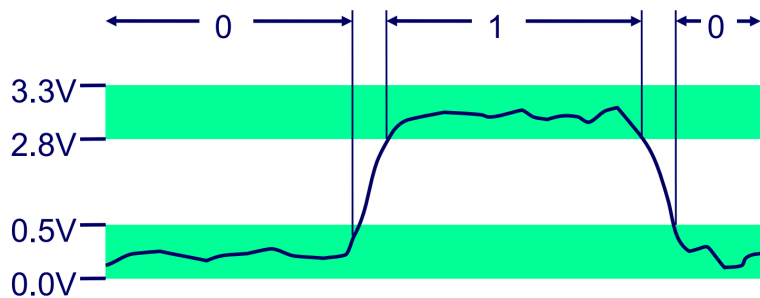
#### 2.1. Base 2 number Representation

- represent  $15123_{10}$  as  $11101101101101_2$

Why computers use binary?

We can use electricity volts to determine the state of a bit. For example, 0V represents 0 and 5V represents 1. It is easy to distinguish between 0 and 1.

#### ■ Why Computers Use Binary?



Main cause: Binary is the most practical system to use!

#### 2.2. Encoding Byte Values

- Byte = 8 bits
- Binary  $00000000_2$  to  $11111111_2$
- Decimal:  $0_{10}$  to  $255_{10}$
- Hexadecimal  $00_{16}$  to  $FF_{16}$

#### 2.3. Different Data types have different lengths.

| 数据类型      | 16 位系统 | 32 位系统 | 64 位 Linux/macOS | 64 位 Windows |
|-----------|--------|--------|------------------|--------------|
| char      | 1      | 1      | 1                | 1            |
| short     | 2      | 2      | 2                | 2            |
| int       | 2      | 4      | 4                | 4            |
| long      | 4      | 4      | 8                | 4            |
| long long | 8      | 8      | 8                | 8            |
| float     | 4      | 4      | 4                | 4            |

<sup>1</sup>Because the slides are basically in English, I will also use English to write the notes, and I will also mix some Chinese into the notes.

| 数据类型   | 16 位系统 | 32 位系统 | 64 位 Linux/macOS | 64 位 Windows |
|--------|--------|--------|------------------|--------------|
| double | 8      | 8      | 8                | 8            |
| 指针     | 2      | 4      | 8                | 8            |
| size_t | 2      | 4      | 8                | 8            |

## 2.4. Boolean Algebra

- And, &
- Or, |
- Not, ~
- Xor, ^

### And

- $A \& B = 1$  when both  $A=1$  and  $B=1$

| & | 0 | 1 |
|---|---|---|
| 0 | 0 | 0 |
| 1 | 0 | 1 |

### Not

- $\sim A = 1$  when  $A=0$

| ~ |   |
|---|---|
| 0 | 1 |
| 1 | 0 |

### Or

- $A | B = 1$  when either  $A=1$  or  $B=1$

|   | 0 | 1 |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 1 |

### Exclusive-Or (Xor)

- $A \wedge B = 1$  when either  $A=1$  or  $B=1$ , but not both

| ^ | 0 | 1 |
|---|---|---|
| 0 | 0 | 1 |
| 1 | 1 | 0 |

## 2.5. Bit-Level Operations in C

(Need to check the slides for details)

可以按位进行计算。

- $\sim 01000001_2 \rightarrow 10111110_2$
- $\sim 0x00 \rightarrow 0xFF$

## 2.6. Logic Operations in C

Compare with the bit level operations

- &&, ||, !
  - ▶ view 0 as false
  - ▶ Anything nonzero as “True”
  - ▶ Always return 0 or 1

## 2.7. Shift operations

- Left shift: <<
  - ▶ shift bit-vector  $x$  left  $y$  positions, throw away extra bits on left.
  - ▶ example:  $01100010 \ll 3 = 00010000$
- Right shift: >>
  - ▶ shift bit-vector  $x$  right  $y$  positions, throw away extra bits on right.
  - ▶ example:  $01100010 \gg 3 = 00001100$
- Undefined behavior: shift by negative number or shift by more than the length of the bit-vector.

### 3. Encoding integers

Unsigned:

$$\text{B2U}(X) = \sum_{i=0}^{w-1} x_i \cdot 2^i$$

negative number?

- Early: 原码表示, 但是 0 存在正的或者负的

Two's Complement: 负数的表示方法, 现代计算机中使用的就是这种方法。

$$\text{B2T}(X) = -x_{w-1} \cdot 2^{w-1} + \sum_{i=0}^{w-2} x_i \cdot 2^i$$

The first bit is the sign bit. 0 means positive, 1 means negative.

有无符号的关键是解释方式的不同, 存储的具体的数据可能是一样的。

### 4. Why should I use Unsigned?

Don't use without understanding implications

```
unsigned i;
for (i = cnt - 2; i >= 0; i--)
    a[i] += a[i + 1];
```

无符号数恒大于等于 0, 因此这个循环的终止条件永远无法满足。当 i 从 0 再减 1 时, 它不会得到负数, 而会按模  $2^w$  回绕到最大的无符号数。

```
#define DELTA sizeof(int)
int i;
for (i = CNT; i - DELTA >= 0; i -= DELTA) {
    // 循环体
}
```

这个错误更加隐蔽: sizeof 的结果类型是无符号的 size\_t, 所以表达式 i - DELTA 会先把 i 转成无符号数, 再进行比较, 条件同样恒为真。

若确实需要用无符号数倒序遍历, 可以利用无符号数的模运算性质:

```
size_t i;
for (i = cnt - 2; i < cnt; i--)
    a[i] += a[i + 1];
```

无符号数适合用于模运算、多精度运算, 以及用位向量表示集合; 除此以外应谨慎使用, 尤其要留意有符号数和无符号数混合运算时发生的隐式转换。

## 5. 内存中的数据表示

### 5.1. 按字节编址

内存可以抽象成一个很大的字节数组, 每个字节都有唯一的地址。地址类似数组下标, 而指针变量保存的就是地址。

- 一个字节由 8 个二进制位组成, 是内存的最小寻址单位。
- 多字节对象占用一段连续的字节, 其地址通常指第一个字节的地址。
- 操作系统为每个进程提供私有的虚拟地址空间。因此, 不同进程中相同的虚拟地址通常对应不同的实际存储位置。

## 5.2. 字长

字长是机器处理整数和地址时采用的标称位数。32 位机器的字长通常为 4 字节，64 位机器的字长通常为 8 字节。

- 32 位地址最多有  $2^{32}$  种取值，因此理论可寻址空间为  $2^{32}$  字节，即 4 GiB。
- 64 位地址理论上可表示  $2^{64}$  个字节，约为 18 EB；实际机器通常只使用其中一部分地址位。
- 机器仍然支持多种长度的数据格式，并非所有数据都等于一个机器字。例如，在 x86-64 中，int 通常仍为 4 字节，而指针通常为 8 字节。

对于按字对齐的对象，相邻 32 位字的起始地址通常相差 4，相邻 64 位字的起始地址通常相差 8。

## 5.3. 字节序

字节序规定一个多字节对象内部的各个字节如何排列在内存中。

- 大端序：最高有效字节放在最低地址，最低有效字节放在最高地址。
- 小端序：最低有效字节放在最低地址，最高有效字节放在最高地址。

设一个 4 字节整数的值为 0x01234567，起始地址为 0x100：

| 字节序 | 0x100 | 0x101 | 0x102 | 0x103 |
|-----|-------|-------|-------|-------|
| 大端序 | 01    | 23    | 45    | 67    |
| 小端序 | 67    | 45    | 23    | 01    |

x86 和 x86-64 通常采用小端序；网络协议通常采用大端序，即“网络字节序”。字节序不会改变对象的数值，只改变各字节在内存中的排列。

## 5.4. 整数在内存中的表示

十进制数 15213 的十六进制表示为 0x3B6D。存入 32 位 int 后，完整位模式为 0x00003B6D：

- 小端序中，从低地址到高地址依次是 6D 3B 00 00；
- 大端序中，从低地址到高地址依次是 00 00 3B 6D。

负数使用补码表示。对于 32 位整数：

$$15213 = 0x00003B6D, \quad -15213 = 0xFFFFC493$$

因此，-15213 在小端序中排列为 93 C4 FF FF，在大端序中排列为 FF FF C4 93。应区分三个概念：整数的数值、它的补码位模式，以及组成该位模式的字节在内存中的顺序。

## 5.5. 逐字节观察对象

可以先把任意对象的地址转换为 unsigned char \*，再把它视为字节数组。unsigned char 恰好占一个字节，因此适合检查对象的原始表示。

```
typedef unsigned char *pointer;

void show_bytes(pointer start, size_t len) {
    for (size_t i = 0; i < len; i++)
        printf("%p\t0x%.2x\n", (void *) (start + i), start[i]);
    printf("\n");
}
```

在小端序 x86-64 机器上执行：

```
int a = 15213;
show_bytes((pointer)&a, sizeof(a));
```

通常会依次看到 6d、3b、00、00。其中 %p 用于输出指针，%x 用于输出十六进制整数。

## 5.6. 指针的表示

指针本身也是一个多字节数据对象，保存某个对象的地址。例如：

```
int b = -15213;
int *p = &b;
```

变量 `b` 中保存 -15213 的补码表示，而指针 `p` 中保存 `b` 的地址。不同机器、编译器和运行环境可能把对象放在不同位置；由于地址空间布局随机化，同一程序多次运行时得到的地址也可能不同。因此，不应假设普通对象每次运行都有固定地址。

## 5.7. 字符串的表示

C 字符串是以空字符结尾的字符数组，每个字符通常使用 ASCII 编码。例如：

```
char s[6] = "18213";
```

它在内存中的字节为：

|     |     |     |     |     |      |
|-----|-----|-----|-----|-----|------|
| '1' | '8' | '2' | '1' | '3' | '\0' |
| 31  | 38  | 32  | 31  | 33  | 00   |

表中第二行是对应的十六进制字节。字符 '0' 的 ASCII 编码是 0x30，所以数字字符 `i` 的编码为 0x30 + `i`。字符串由单字节字符组成，不需要交换字符内部的字节，因此通常不受大小端影响。

## 5.8. 阅读按小端序排列的机器代码

反汇编把二进制机器代码转换成可读的汇编指令。在小端机器中，多字节立即数在机器代码里会按小端序排列。

例如指令中的数值 0x12AB 若按 32 位保存：

|          |             |
|----------|-------------|
| 补足到 32 位 | 00 00 12 AB |
| 小端序存入内存  | AB 12 00 00 |

因此，在反汇编的指令字节中看到 `ab 12 00 00` 时，应把它还原为数值 0x000012AB，而不是按显示顺序直接读成 0xAB120000。

## 6. 本讲总结

- 所有信息最终都表示为二进制位，字节是最小寻址单位。
- C 提供按位运算、逻辑运算和移位运算；必须区分它们的语义。
- 同一组位既可以按无符号数解释，也可以按补码有符号数解释。
- 固定位宽整数的加法和乘法会截断高位，本质上具有模  $2^w$  的性质，因此可能发生溢出。
- 有符号数与无符号数混合运算时，隐式转换可能产生违反直觉的结果。
- 多字节对象在内存中的排列受字节序影响；x86-64 通常使用小端序。
- 指针保存地址，字符串是以 0 字节结尾的字符数组。